

ODC v4

Audit Report

Security, reliability, and operability audit

Date: 2026-09-25

Classification: Production readiness review

Executive Summary

ODC v4 is an **ACAMR-9 (Adaptive Self-Refinement)** LLM agent. It implements:

- 13 OSINT endpoints (flights, earthquakes, CVE, bitcoin, crypto prices, weather, etc.)
- Persistent memory with cross-session lineage (5-step ritual)
- Injection-resistant framing (PAM blocks)
- 6-tier Evidence Taxonomy (calibrated confidence)
- DEEP-REASON System 2 (5-lens council, multi-hypothesis, analogical)
- **Level 9 self-refinement engine** (field-grounded, constitutional, sandbox-verified)

This audit was triggered by a self-identified concern about the correctness of the self-refinement sandbox simulation. That concern was validated — two real bugs were found and fixed.

Audit verdict

Dimension	Status
Self-refinement correctness	■ FIXED (2 bugs identified + 9 known scenarios)
Reproducibility (install, tests)	■ FIXED (lockfile, CI, network/llm markers)
Operational SLOs	■ FIXED (budgets, percentiles, retry policies)
Authorization model	■ FIXED (workspace roots, web allow-list, capability tokens, immutable audit)
Dynamic execution isolation	■ FIXED (default OFF, subprocess + container runners, deprecation warning)
Agent quality evaluation	■ FIXED (offline fixtures, regression scenarios)

372 unit tests passing (up from 310 before the audit), with a dedicated `test_sandbox_scenarios.py` suite (9 known-good scenarios) that would have caught the original bugs.

Remaining gaps

Gap	Risk	Mitigation
ContainerRunner untested in CI (no Docker)	medium	Add CI service container or mock
Capability token issuance not yet integrated with web UI	low	Out of scope for this audit
Audit DB is local SQLite (no remote attestation)	low	OK for single-tenant, add remote attestation for multi-tenant

2. Reproducibility

2.1 Issues identified

2.1.1 README referenced the wrong install command

The README said:

```
pip install -r requirements.txt
```

...but the actual dependency manifest was in `pyproject.toml`. The `requirements.txt` file did not exist (or contained outdated entries), making the documented install procedure unreliable.

Fix: generated `requirements.txt` from the resolved `venv` (`pip freeze`) so the documented procedure works. Also created `requirements-lock.txt` for full reproducibility.

2.1.2 No CI

There was no automated pipeline. "310 tests passing" was a claim that needed manual re-execution each time.

Fix: created `.github/workflows/ci.yml` with three jobs:

- **lint** — `ruff check + ruff format --check`
- **unit-tests** — runs the unit suite across Python 3.10, 3.11, 3.12
- with coverage reporting
- **security-tests** — runs injection/framing/constitutional tests in
- isolation
- **network-tests** — runs live API tests on schedule only (since
- public APIs are flaky)

2.1.3 Network tests not separated

Live-API tests (`test_osint.py`, `test_solid_benchmark.py`) were mixed with unit tests. They are flaky when the sandbox has limited connectivity, causing false CI failures.

Fix: added `network` and `llm` pytest markers. Network tests are now opt-in via the schedule workflow.

2.2 Updated install procedure

```
# Reproducible install
python -m pip install -r requirements-lock.txt

# Or for full dev environment
pip install -e ".[dev,nvidia]"
```

2.3 CI matrix

Job	Python	Frequency	Caches
lint	3.11	every push	pip
unit-tests	3.10 / 3.11 / 3.12	every push	pip (lockfile hash)
security-tests	3.11	every push	—
network-tests	3.11	nightly + manual	—

4. Authorization model

4.1 Issues identified

The previous design had:

- `fs.read` accepting absolute paths and reading any file the process
- could access (no workspace root enforcement).
- `fs.write` allowing creation/overwrite of any path.
- `web.fetch` calling any URL with no allow-list.
- Dynamic tool execution with no isolation (see §5).
- No immutable audit trail.

This meant: anyone with `confirm=True` on the prompt could read `/etc/passwd` or call `https://internal-api.example.com`.

4.2 Implementation

`odc/governance/authz.py`:

4.2.1 WorkspacePolicy

```
@dataclass
class WorkspacePolicy:
    allowed_roots: list[Path]
    deny_patterns: list[str] # e.g., [".pem", "id_rsa"]
    max_file_size_mb: float
```

A path is allowed iff it resolves to a subdirectory of one of the `allowed_roots`. Symlinks are resolved before the check. Pattern denylist applies after the root check.

4.2.2 WebPolicy

```
@dataclass
class WebPolicy:
    allowed_domains: list[str] # ["api.example.com"]
    denied_domains: list[str] # ["malicious.com"]
    allowed_schemes: tuple # ("http", "https")
    deny_private_ips: bool # 127.x, 10.x, 192.168.x, etc.
```

`file://`, `javascript:`, and other dangerous schemes are rejected. Private IP ranges are denied by default to prevent SSRF.

4.2.3 CapabilityToken

HMAC-signed token with `user`, `tool` (or `"*"`), `scope`, `exp`, `iat`. Verifies on every tool call. Expired tokens rejected. Tampered tokens rejected (signature mismatch).

4.2.4 AuthorizationGuard

Combines `WorkspacePolicy` + `WebPolicy` + `CapabilityToken`. Returns `AuthDecision` with `allowed/reason/paths/domains`.

4.2.5 AuditTrail

Append-only SQLite with **HMAC-chained entries**: `entry_hash = hmac(secret, prev_hash || entry_payload)`. Tampering with any entry breaks the chain at that point (detectable by `verify_chain()`).

4.3 Test coverage

`tests/test_authz.py` — 24 tests:

- Workspace allows/denies by root
- Workspace blocks `/etc/passwd` outside root
- Workspace deny patterns (`.pem`)
- Workspace max file size
- Web allows allowlisted domain + subdomain
- Web denies unknown domain
- Web denies private IPs (127.x, 10.x)
- Web denies `file://` and `javascript:`
- Web denies explicit deny domains
- CapabilityToken sign/verify
- CapabilityToken wrong secret fails
- CapabilityToken expired fails
- CapabilityToken tampered payload fails
- AuthorizationGuard path allowed/blocked
- AuthorizationGuard URL allowed
- Capability wildcard (*)
- Capability specific tool only
- AuditTrail hash chain valid
- AuditTrail detects tampering
- AuditTrail query by user/tool/decision

6. Agent quality evaluation

6.1 What was missing

Unit tests confirm code works. They do not prove the agent is reliable with users. The audit added:

6.2 Implementation

`tests/test_quality_eval.py` — 22 tests using deterministic fixtures (no live network).

6.2.1 Tool selection precision

5 parametrized tests verify that queries route to the right tool:

```
"Bitcoin block height" → osint.bitcoin
"flights over Europe" → osint.flights
"temperature in São Paulo" → osint.weather
```

"earthquakes today" → osint.earthquakes
"Bitcoin price USD" → osint.crypto_prices

6.2.2 Completeness

3 tests verify responses include required fields (height for Bitcoin, °C for weather, callsign for flights).

6.2.3 Citation accuracy

2 tests verify:

- USGS URLs match the real pattern
- Fixtures contain only known real domains (no invented URLs)

6.2.4 Honesty calibration

1 test verifies that a low-tier + low-evidence claim does NOT inflate to high confidence.

6.2.5 Injection resistance

5 parametrized tests verify that real-world injection attacks (ignore-instructions, role-marker, ###-System, [INST]) are neutralized by the framing layer.

6.2.6 Recovery

2 tests verify that 503 and timeout failures from external services are handled gracefully (don't crash the agent loop).

6.2.7 Authorization enforcement

3 tests verify the new guard rejects paths outside workspace, private IPs, and detects audit tampering.

6.3 Honest limitations

- Tests use **deterministic fixtures**, not live LLM calls. They test
- the surrounding code (framing, prompt routing, authorization) but
- not the LLM itself.
- Behavioral regression (when changing models or prompts) requires
- separate evaluation with real LLM calls. The `test_real_llm_benchmark.py`
- suite provides 4 such tests but requires a working NVIDIA API key.
- Continuous red-teaming and human evaluation are out of scope for
- this audit.

6.4 Recommended next steps

Action	Priority	Effort
Add behavioral regression suite (prompt vs prompt)	P2	1 week

Action	Priority	Effort
Continuous red-teaming (adversarial prompts)	P2	ongoing
Human evaluation (graded task set)	P2	2 weeks
Cost/latency benchmarks per task type	P2	1 week
Multi-turn conversation quality tests	P2	1 week

8. Recommendations for production deployment

8.1 Before any deployment

1. **Enable ContainerRunner** for all dynamic tool execution. SubprocessRunner is fallback-only. 2. **Configure WorkspacePolicy** with explicit allowed roots per tenant. 3. **Configure WebPolicy** with explicit allow-list per tenant. 4. **Issue CapabilityTokens** at session start with short TTL (1 hour). 5. **Set TaskBudget** per user/tenant class.

8.2 Monitoring

1. Wire LatencyTracker, CostTracker, ToolFailureTracker into the agent loop. 2. Send metrics to Prometheus or similar. 3. Alert on SLO violations (see §3.3).

8.3 Audit

1. Keep AuditTrail enabled for all tool calls. 2. Run verify_chain() periodically to detect tampering. 3. Retain audit logs for compliance window (e.g., 1 year).

8.4 Continuous

1. Run test_quality_eval.py in CI on every commit. 2. Run test_real_llm_benchmark.py nightly (requires API key). 3. Run live network tests (test_osint.py) weekly. 4. Add new injection patterns to test_quality_eval.py::TestInjectionResistance as they appear in the wild.